

Tutorial 03: A More Complex Shader - Glass with Transparency and Shadows

⚠ **Build issues?** See [Troubleshooting](#) at the end of this tutorial for help reading build errors from the terminal.

In Tutorials 01 and 02 you wrote an unlit shader and extended it with a custom bind group. Now you'll write a shader that actually interacts with the scene: a **glass shader** that blends with the background, catches a specular glint from the sun, brightens at its silhouette like real glass, and **receives shadows** from the objects around it.

Along the way you'll learn how the engine decides *which render pass* draws a material - and why transparent and custom-shader materials take a different path through the frame than the opaque PBR objects.

What you'll learn: - The Scene bind group (@group(1)): lights, shadow maps, and how to #include them - View-vector math: fresnel-style rim lighting from the camera position - A Blinn-Phong specular highlight from the directional light - Sampling shadows with the engine's shared calculate_shadow() (cascades, bias, PCF) - Transparency: alpha output, the Transparent feature flag, and alpha blending - Why transparent and custom-shader materials render in the **forward pass**, sorted back-to-front

What's provided: - The complete glass shader at examples/tutorial/assets/shaders/glass.wgsl - unlike Tutorial 01 there is nothing to type in; this tutorial walks through the finished file section by section (the // Tutorial 03 - Step N markers match the steps below) - The C++ setup in examples/tutorial/main.cpp (Tutorial 03 Glass Shader region): shader registration, glass material, and a second boat model - One commented-out material assignment you'll uncomment at the end (Step 12)

Step 1: Introduction - Two Render Paths

Before touching the shader, understand where it will run.

The engine renders opaque geometry through a **deferred** pipeline: the GBufferPass writes surface attributes (albedo, normals, roughness...) into a set of textures, and the CompositionPass computes lighting once per pixel from those textures. This is fast, but it has two hard limitations:

1. **The G-buffer stores exactly one surface per pixel.** A transparent surface needs the background *behind* it to already be shaded so it can blend with it - there is no slot for "the surface and also what's behind it".
2. **The G-buffer path runs one fixed shader.** Your custom WGSL never executes there; the pass only knows how to write the standard attribute layout.

So the engine routes two kinds of materials through the **forward** path instead (ForwardTransparencyPass, which runs *after* the deferred composition and the skybox, when the lit background image already exists):

```
// include/engine/rendering/Material.h
bool usesForwardShading() const
{
    return isTransparent() || (!m_shader.empty() && m_shader != shader::defaults::PBR)
}
```

- **Transparent materials** - blending needs the shaded background underneath.
- **Custom-shader materials** (any shader that isn't the default PBR_Lit_Shader) - the deferred path would ignore their shader. Opaque custom materials still get depth writes and no blending, so they render correctly; they just draw in the forward pass.

Our glass material is **both**: it uses a custom shader *and* it is transparent. GBufferPass skips it (GBufferPass.cpp, the usesForwardShading() check) and ForwardTransparencyPass picks it up.

Why back-to-front? Alpha blending is order-dependent: each blended surface mixes with whatever is already in the framebuffer. The pass sorts its draw candidates by squared distance to the camera and draws the farthest first (ForwardTransparencyPass.cpp, the DrawCandidate sort), so nearer glass composites over farther glass over the opaque background.

Building the Project

```
# Windows (Dawn backend - same as the VS Code tasks)
scripts\build-example.bat tutorial Debug DAWN
```

```
# Linux
bash scripts/build-example.sh tutorial Debug DAWN
```

Where the build lands: the build script uses **per-backend build directories** on Windows. DAWN (vendored prebuilt, recommended) builds into examples\build\tutorial\Windows\Debug-DAWN, WGPU into ... \Debug-WGPU, and Dawn built from source (DAWN ... SOURCE) into the plain ... \Debug folder. On Linux/Mac the directory is always examples/build/tutorial/<Linux|Mac>/Debug.

Build once now. Even before Step 12 the program runs: you'll see the familiar scene plus a **second fourareen boat** behind the first one - still rendered with its regular PBR material, because the glass material assignment is commented out.

Step 2: Engine Bind Groups via #include

Open examples/tutorial/assets/shaders/glass.wgsl and find // Tutorial 03 - Step 2.

```
#include "engine://core/frame_uniforms.wgsl"
#include "engine://core/scene_bindings.wgsl"
```

```
#include "engine://core/object_uniforms.wgsl"
```

```
#include "engine://lib/shadow.wgsl"
```

Frame (@group(0), u_frame) and Object (@group(3), u_object) are the same includes you used in Tutorials 01 and 02. Two includes are new:

engine://core/scene_bindings.wgsl - the **Scene bind group at @group(1)**, the slot the unlit shader left empty. It is auto-generated (like the Frame/Object structs) and declares, among others:

Binding	Name	What it is
0	u_lights	Storage buffer with every light in the scene (LightStruct array + count)
1	shadow_sampler	A comparison sampler - the GPU compares a reference depth against the shadow map for you
2	shadow_maps_2d	Depth texture array holding the directional/spot shadow maps (CSM cascades are slices)
4	u_shadows	Per-shadow uniforms: light-space matrix, bias values, PCF kernel size, cascade split
5-12	environment, clusters, IBL	Used by the PBR shader; declared but unused here

Including it is **all the wiring shadows need on the shader side**. The engine builds and binds the Scene group once per frame for every forward-pass draw (ForwardTransparencyPass.cpp passes it to the BindGroupBinder alongside Object and Material) - your shader just declares that it wants it.

engine://lib/shadow.wgsl - a hand-written **library** include (note lib/, not core/): shared shadow-sampling functions, most importantly calculate_shadow(). This is the *exact same code* PBR_Lit_Shader.wgsl and the deferred composition use. Reusing it means your glass gets pixel-identical shadows to the PBR objects next to it - and when someone improves the cascade math later, glass improves with it. It pulls in engine://lib/lighting.wgsl itself (the include resolver deduplicates repeated includes, so shaders can include both without conflicts).

Step 3: Vertex Structures - Passing World-Space Data

Find // Tutorial 03 - Step 3:

```
struct VertexInput {  
    @location(0) position: vec3f,  
    @location(1) normal: vec3f,  
    @location(2) texCoord: vec2f,  
}
```

```
struct VertexOutput {
    @builtin(position) position: vec4f,
    @location(0) worldPosition: vec3f,
    @location(1) worldNormal: vec3f,
}
```

Tutorial 01's unlit shader only forwarded UVs. Glass needs **world-space position and normal** in the fragment shader, because every effect in this tutorial is world-space math:

- The **view vector** (fresnel) is `cameraWorldPosition - worldPosition`
- The **specular** half-vector combines the view vector with the world-space light direction
- The **shadow lookup** projects the world position into the light's clip space

The rasterizer interpolates both across each triangle, so every fragment knows where it sits in the world and which way it faces.

Step 4: Glass Material Uniforms at @group(2)

Find // Tutorial 03 - Step 4:

```
struct GlassMaterialUniforms {
    // rgb = glass tint, a = base opacity when looking straight through
    color: vec4f,
    // x = fresnel strength, y = specular shininess (Blinn-Phong exponent),
    // z = specular strength, w = shadow dimming (0 = shadows invisible,
    // 1 = shadowed glass goes fully dark)
    params: vec4f,
}

@group(2) @binding(0)
var<uniform> glassMaterialUniforms: GlassMaterialUniforms;
```

Like the unlit material in Tutorial 01, the Material group is declared **by hand** - this shader has its own parameter set, not the engine's PBR material. Unlike Tutorial 01 there are no texture bindings: the glass look comes entirely from these eight floats.

The C++ counterpart lives at the top of `main.cpp`:

```
// Tutorial 03 - Step 4: GPU-facing properties for the glass shader.
struct GlassProperties
{
    glm::vec4 color{0.55f, 0.75f, 0.9f, 0.3f};
```

```
    glm::vec4 params{1.0f, 64.0f, 1.0f, 0.6f};
};
static_assert(sizeof(GlassProperties) % 16 == 0, "uniform data must stay 16-byte align
```

The two must match byte for byte (32 bytes: two vec4f). The engine sizes the GPU buffer from the *shader's* reflected struct and uploads the C++ struct's bytes into it (WebGPUMaterial.cpp writes getPropertiesData() / getPropertiesSize() into the material bind group) - a size mismatch means silently truncated or garbage parameters.

Step 5: The Vertex Shader

Find // Tutorial 03 - Step 5:

```
@vertex
fn vs_main(input: VertexInput) -> VertexOutput {
    var output: VertexOutput;

    let worldPosition = u_object.modelMatrix * vec4f(input.position, 1.0);
    output.position = u_frame.viewProjectionMatrix * worldPosition;
    output.worldPosition = worldPosition.xyz;
    // w = 0.0 marks a direction: directions must not receive translation.
    output.worldNormal = normalize((u_object.normalMatrix * vec4f(input.normal, 0.0)).

    return output;
}
```

Same matrix pipeline as Tutorial 01, with two differences:

- We keep the intermediate **world-space position** instead of discarding it after projection (this time using the combined viewProjectionMatrix for the clip-space output).
- The normal is transformed by the **normal matrix**, not the model matrix, and with $w = 0.0$. A normal is a direction: translation must not move it, and under non-uniform scaling the model matrix would bend it away from the true surface direction - the normal matrix (inverse-transpose) corrects for that.

Step 6: Fresnel-Style Rim - View-Vector Math

Find // Tutorial 03 - Step 6 in the fragment shader:

```
let normal = normalize(input.worldNormal);

let viewDirection = normalize(u_frame.cameraWorldPosition - input.worldPosition);
```

```
let facingAmount = max(dot(normal, viewDirection), 0.0);
let fresnel = glassMaterialUniforms.params.x * pow(1.0 - facingAmount, 5.0);
```

First: **re-normalize the normal**. Interpolating unit vectors across a triangle produces vectors slightly shorter than 1, which would subtly darken every dot product.

The **view vector** points from the surface toward the camera - this is why `u_frame.cameraWorldPosition` exists in the Frame uniforms. The dot product with the normal measures how directly the surface faces you:

- $\text{dot} \approx 1$ - you look straight through the glass: it transmits, $\text{rim} \approx 0$
- $\text{dot} \approx 0$ - the surface curves away at the silhouette: it reflects, $\text{rim} \approx \text{max}$

That behavior - reflectance rising steeply at grazing angles - is the **Fresnel effect**, and it's the single strongest visual cue that a material is glass. `pow(1 - dot, 5.0)` is **Schlick's approximation** of the physical curve; it is the same fifth-power term the engine's PBR pipeline uses inside `fresnel_schlick()` (`resources/shaders/lib/lighting.wgsl`), just simplified to a scalar with $F_0 = 0$.

Step 7: Finding the Sun - A Specular Highlight

Find // Tutorial 03 - Step 7:

```
var lightDirection = vec3f(0.0, 1.0, 0.0);
var lightRadiance = vec3f(0.0);
var shadowVisibility = 1.0;
for (var i = 0u; i < u_lights.count; i = i + 1u) {
    if (i >= arrayLength(&u_lights.lights)) {
        break;
    }
    let light = u_lights.lights[i];
    if (light.light_type != 1u) {
        continue;
    }

    lightDirection = get_direction_from_transform(light.transform);
    lightRadiance = light.color * light.intensity;

    shadowVisibility = calculate_shadow(input.worldPosition, normal, light);
    break;
}
```

`u_lights` (from the Scene include) holds **every** light. The PBR shader accumulates all of them through a clustered lookup; glass deliberately keeps it simple and scans for the **first directional light** (`light_type == 1`, matching `engine::rendering::LightType`) - the sun is what produces the readable glint and the

shadow. The `arrayLength` guard exists because `lights` is a runtime-sized array: a count that outran the buffer must never read out of bounds.

`get_direction_from_transform()` (from `lib/lighting.wgsl`) extracts the light's forward axis from its transform and returns the vector **toward** the light - the sign convention all engine lighting helpers share.

The highlight itself (right below the loop):

```
let halfVector = normalize(viewDirection + lightDirection);
let specular = pow(max(dot(normal, halfVector), 0.0), glassMaterialUniforms.params
    * glassMaterialUniforms.params.z
    * shadowVisibility;
```

This is **Blinn-Phong**: the half vector sits halfway between the view and light directions, and the more the normal aligns with it, the closer the surface is to mirror-reflecting the light into your eye. The exponent (`params.y`, 64 by default) controls tightness - higher means a smaller, sharper glint. It's cheaper and easier to reason about than the PBR GGX lobe, which makes it the classic teaching model. Multiplying by `shadowVisibility` kills the glint in shadow: a specular highlight is a direct reflection of the light source, and a shadowed point has no line of sight to it.

Step 8: Received Shadows - What `calculate_shadow()` Does

The single call in Step 7's loop is the whole shadow integration:

```
shadowVisibility = calculate_shadow(input.worldPosition, normal, light);
```

It returns 1.0 for fully lit, 0.0 for fully occluded, and fractional values at PCF-softened edges. You get the engine's full shadow feature set for free, because `lib/shadow.wgsl` is the same code the PBR shader runs. What happens inside (worth reading at `resources/shaders/lib/shadow.wgsl`):

1. **Cascade selection** - directional lights use cascaded shadow maps (CSM): several shadow maps at increasing distances. `select_cascade()` compares the fragment's view-space depth against each cascade's split distance and picks the matching slice of `shadow_maps_2d`.
2. **Light-space projection** - the world position is multiplied by the cascade's `viewProj` matrix (from `u_shadows`), giving the fragment's position *as the light sees it*: XY become shadow-map UVs, Z the depth to compare.
3. **Bias** - a small depth offset prevents *shadow acne* (a surface shadowing itself through depth quantization). The bias is **slope-scaled**: surfaces at grazing angles to the light (low `dot(normal, lightDirection)`) get more bias, which is why `calculate_shadow` takes the normal.
4. **PCF filtering** - instead of one binary lit/shadowed test, a small kernel of neighboring texels is sampled through the **comparison sampler** (`textureSampleCompareLevel` - the hardware performs depth <

storedDepth per sample) and averaged. That average is what makes shadow edges soft instead of aliased staircases.

A limitation to know about: the glass *receives* correct shadows, but it also still *casts* a fully opaque one. The shadow pass renders casters into a depth-only map - depth has no concept of alpha, so from the sun's perspective the glass boat is as solid as any other mesh. Handling translucent casters (colored/partial shadows) requires additional machinery that is out of scope here.

Step 9: Composing Transparency

Find // Tutorial 03 - Step 9:

```
let litFactor = mix(1.0 - glassMaterialUniforms.params.w, 1.0, shadowVisibility);
let bodyColor = glassMaterialUniforms.color.rgb * litFactor;
let rimColor = vec3f(1.0) * fresnel * litFactor;
let specularColor = lightRadiance * specular;

let alpha = clamp(glassMaterialUniforms.color.a + fresnel + specular, 0.0, 1.0);

return vec4f(bodyColor + rimColor + specularColor, alpha);
```

Three decisions worth understanding:

Shadow dimming instead of blackout. Glass in shadow still transmits the background - it shouldn't turn into a black cutout. So the shadow scales the glass's *own* contribution (tint + rim) down to 1 - shadowDimming (default 0.6 → 40% brightness) instead of multiplying everything toward zero. The background showing through is untouched; it was already shaded, shadows included, by the deferred passes.

Alpha rises at the rim and under the highlight. The alpha you return feeds the blend equation $\text{result} = \text{shaderColor} \times \text{alpha} + \text{background} \times (1 - \text{alpha})$. Physically, glass at grazing angles reflects more and transmits less - so the fresnel term raises opacity toward the silhouette. The specular term is added too so the glint doesn't get washed out by the blend. (The PBR shader solves this exactly with a fresnel-corrected effective alpha - see the derivation comment at the end of PBR_Lit_Shader.wgsl; ours is the simplified version of the same idea.)

Where does the blending come from? Not from the shader - WGSL cannot express blend state. It comes from the pipeline, which brings us to the C++ side.

Step 10: Register the Glass Shader

Open examples/tutorial/main.cpp and find // Tutorial 03 - Step 10 (already in place):


```

engine::rendering::webgpu::ShaderDescriptor glassShader;
glassShader.name          = "glass";
glassShader.type          = engine::rendering::ShaderType::Custom;
glassShader.path          = PathProvider::getShaders("glass.wgsl");
glassShader.vertexLayout = engine::rendering::VertexLayout::PositionNormalUV;
shaderRegistry.registerShader(shaderFactory.buildFromDescriptor(glassShader));

```

Compare this to Tutorial 01's unlit descriptor: this one is even smaller - **no groups metadata at all**.

Reflection recovers everything:

- Frame (@group(0)), Scene (@group(1)) and Object (@group(3)) come from the engine #includes and fall back to their canonical engine roles automatically
- The Material group (@group(2)) is a single uniform buffer with no texture slots, so there are no texture-slot bindings to describe (the unlit shader needed metadata only to map its texture binding to the DIFFUSE slot)

When you run, the log confirms it: WebGPUShaderFactory: built 'glass' from descriptor with 4 bind groups.

Step 11: Create the Material and Set the Transparent Flag

Find // Tutorial 03 - Step 11:

```

auto glassProperties = GlassProperties{};
auto maybeGlassMaterial = resourceManager->m_materialManager->createMaterial(
    "Glass_Material",
    glassProperties,
    "glass",
    {} // no textures - the look comes entirely from the uniform parameters
);
...
auto glassMaterial = maybeGlassMaterial.value();
glassMaterial->setFeatureMask(glassMaterial->getFeatureMask() | engine::rendering::Mat

```

createMaterial() is the same call Tutorial 01 used - the properties struct is your own GlassProperties, the shader name is "glass", and the texture map is empty.

The new part is the **feature mask**. MaterialFeature::Flag (see include/engine/rendering/MaterialFeatureMask.h) is a bitmask describing material traits; createMaterial derives texture-related flags automatically, but *transparency is a deliberate choice you make*, so you set it yourself - OR-ing it into the existing mask rather than overwriting.

What the flag actually does - follow it through the engine:

1. `Material::isTransparent()` checks the flag, so `usesForwardShading()` returns true → `GBufferPass` skips the material, `ForwardTransparencyPass` draws it (Step 1).
2. When the forward pass asks the pipeline manager for a pipeline (`WebGPUPipelineManager::getOrCreatePipeline(mesh, material, passContext)`), it translates the flag into `blendEnabled = true` for the pipeline key.
3. The pipeline factory (`WebGPUPipelineFactory::createRenderPipeline`) then builds the pipeline with **SrcAlpha / OneMinusSrcAlpha color blending** and - critically - **depth writes off**. Depth *testing* stays on (glass hidden behind an opaque wall is still culled), but a translucent surface must not write depth: it would occlude other translucent surfaces drawn after it, breaking the back-to-front alpha math.

One pipeline per (shader, mesh topology, cull mode, blend, formats) combination is created once and cached - subsequent frames reuse it.

Step 12: The Glass Boat - Assign and Run

Find // Tutorial 03 - Step 12:

```
auto maybeGlassModel = resourceManager->m_modelManager->createModel(  
    PathProvider::getAssets("fourareen.obj"),  
    "fourareen_glass"  
);
```

Why the explicit name? `ModelManager::createModel` dedups by name, and the name defaults to the file path - without "fourareen_glass" you'd get the *same* model instance the PBR boat uses, and assigning the glass material would repaint both boats.

Now uncomment the material assignment:

```
// Uncomment this line:  
glassModel->getSubmeshes()[0].material = glassMaterial->getHandle();
```

([0] because fourareen has a single submesh, same as Tutorial 01's plane.)

The node below places the glass boat at (1.2, 1.0, -2.0) - behind the PBR boat relative to the sun, so the opaque boat's shadow falls **across the glass**, which is exactly where you can watch the received shadow dim it.

Rebuild and Run

```
scripts\build-example.bat tutorial Debug DAWN  
examples\build\tutorial\Windows\Debug-DAWN\Tutorial.exe
```

(If you built with the WGPU backend instead, the executable is in
examples\build\tutorial\Windows\Debug-WGPU\.)

VS Code shortcuts: - Press F5 to build and run with debugger - Or open **Run and Debug** panel
(Ctrl+Shift+D) → select "**Tutorial (Debug) - Windows**" (or your platform)

Expected Result

- ☒ **A translucent blue-tinted boat** behind the PBR boat - the floor and background show through it
- ☒ **Bright silhouette edges** where the hull curves away from the camera (fresnel rim); fly the camera around it (WASD + mouse) and watch the rim follow the silhouette
- ☒ **A specular glint** from the sun on surfaces angled between camera and light
- ☒ **Dimmed glass where the opaque boat's shadow crosses it** - the tint and rim darken while the background keeps showing through
- ☒ The glass boat still **casts** a regular opaque shadow on the floor (see the limitation note in Step 8)

In the log you should find:

```
WebGPUShaderFactory: built 'glass' from descriptor with 4 bind groups  
Registered shader 'glass'
```

and with debug logging (`spdlog::set_level(spdlog::level::debug)`), each frame reports:

```
ForwardTransparencyPass: drew 2 transparent items (0 skipped)
```

(2, not 1: your Tutorial 02 floor uses the custom unlit shader, so `usesForwardShading()` routes it through the forward pass too - the glass boat is the second item.)

Understanding What We Built

The material decides the render path. No pass list to edit, no flags on the node - the routing is a pure function of the material: `usesForwardShading()` = transparent OR custom shader. The same predicate excludes the item from the G-buffer and includes it in the forward pass, so an object can never be drawn twice or not at all.

The frame, from the glass boat's perspective:

1. ShadowPass - renders casters (including the glass boat - depth only) into the shadow map slices
2. GBufferPass - skips the glass (`usesForwardShading()`), draws everything opaque into the G-buffer
3. CompositionPass - shades the opaque scene, shadows included -> lit HDR image
4. SkyboxPass - fills the background

5. ForwardTransparencyPass

- └ collects items whose material usesForwardShading()
- └ sorts them back-to-front by camera distance
- └ gets/creates a pipeline per material (blend + no depth-write from the Transparent flag)
- └ binds Frame, Scene (lights + shadow maps), Material, Object
- └ draws - your glass.wgsl runs here, over the finished background

One shadow implementation, everywhere. The Scene bind group and lib/shadow.wgsl mean a custom shader opts into engine shadows with one include and one function call - and stays correct when the engine's shadow internals change.

Experiments to Try

1. Tint and opacity - in main.cpp:

```
glm::vec4 color{0.9f, 0.3f, 0.3f, 0.15f}; // barely-there red glass
```

2. Frosted vs polished - lower the shininess for a broad soft sheen, raise it for a hard glint:

```
glm::vec4 params{1.0f, 8.0f, 0.5f, 0.6f}; // frosted  
glm::vec4 params{1.0f, 256.0f, 2.0f, 0.6f}; // polished
```

3. Exaggerate the shadow - set shadow dimming (params.w) to 1.0 and the shadowed glass goes fully dark; set it to 0.0 and shadows stop affecting the glass entirely - toggle between them to convince yourself the shadow sampling works.

4. Visualize the fresnel term - temporarily return it as a color to see the rim mask in isolation:

```
return vec4f(vec3f(fresnel), 1.0);
```

5. Animate the tint - u_frame.time is right there:

```
let pulse = 0.5 + 0.5 * sin(u_frame.time);  
let bodyColor = glassMaterialUniforms.color.rgb * litFactor * pulse;
```

Key Takeaways

-  **Scene bind group** - #include "engine://core/scene_bindings.wgsl" brings lights + shadow maps into any shader at @group(1)

-  **Shared shadow code** - `calculate_shadow()` from `engine://lib/shadow.wgsl` = the PBR shader's cascades, bias, and PCF for free
 -  **View-vector math** - fresnel rim from `cameraWorldPosition`, Blinn-Phong glint from the half vector
 -  **Transparent flag** - one material bit → alpha blending + depth-write-off in the pipeline
 -  **Forward routing** - transparent and custom-shader materials draw in `ForwardTransparencyPass`, back-to-front, over the finished deferred image
 -  **Struct contract** - the C++ properties struct and the WGSL material struct must match byte for byte
-

What's Next?

In **Tutorial 04**, you'll move from writing shaders to writing a **render pass**: implementing a post-processing vignette effect, recording its commands, and integrating it into the renderer's frame graph.

Next Tutorial: [04_postprocessing.md](#)

Previous Tutorial: [02_custom_bindgroup.md](#)

Further Reading

- [WebGPU WGSL Specification](#)
 - [Engine Bind Group System](#)
 - Engine shadow library: `resources/shaders/lib/shadow.wgsl`
 - Forward pass source: `src/engine/rendering/ForwardTransparencyPass.cpp`
 - [LearnWebGPU Tutorial](#)
-

Troubleshooting

Build Failures - Reading Terminal Output

 **Important:** When using `scripts\build-example.bat` (directly or via the VS Code task), the task system may report success even if the build actually failed. You **MUST check the terminal output** to see the real result.

What to look for in terminal: 1. Scroll to the **very end** of the terminal output 2. Look for `[SUCCESS]` Example `'tutorial' built successfully!` - if this appears, build succeeded 3. If you see `[ERROR]` Build failed. - the build failed regardless of task status

Common build issues: - **CMake cache** - Delete the example's build folder (`examples\build\tutorial`) then rebuild clean - **C++ errors around the feature mask** - the OR must combine two `MaterialFeature::Flag` values; make sure you kept `glassMaterial->getFeatureMask() | ...` rather than assigning the flag alone

Shader Issues

Shader errors surface at **runtime**, not compile time - the shader is loaded, include-expanded, validated and reflected when the program starts. Check the console / `run_out.log`; in Debug builds the validator fails fast with the offending line and column.

"could not read shader 'glass'" - The path must be `PathProvider::getShaders("glass.wgsl")` and the file must be in `examples/tutorial/assets/shaders/` - the build copies the assets folder next to the executable

Redeclaration errors mentioning `LightStruct`, `FrameUniforms`, or `shadow functions` - Don't hand-write structs the includes already provide, and don't paste code from `lib/shadow.wgsl` into your shader - include it

Glass renders but is fully opaque - The Transparent feature flag wasn't set: without it the pipeline builds with blending off. Verify the `setFeatureMask` line runs *after* `createMaterial` (which overwrites the mask with texture-derived flags)

Glass is invisible - Alpha may be composing to `~0`: check `color.a` isn't 0 with `fresnelStrength` also 0 - Verify the material assignment (Step 12) is uncommented and the model was created with the distinct `"fourareen_glass"` name

Both boats turned to glass - The second `createModel` call is missing its explicit name, so it returned the cached first model - see Step 12

No shadow visible on the glass - The glass boat must sit in another object's shadow; move it (or the sun) so the opaque boat's shadow crosses it - Set `params.w` (shadow dimming) closer to `1.0` to make the effect unmistakable

Wrong or garbage parameter values in the shader - `GlassProperties` (C++) and `GlassMaterialUniforms` (WGSL) sizes diverged - both must be exactly two `vec4` (32 bytes)

Debug Strategy

If errors are unclear: 1. Run the tutorial - include expansion, reflection, validation and pipeline creation all happen at load time and log with line/column 2. Add `spdlog::set_level(spdlog::level::debug);` after `engine.initialize(options);` to see per-frame pass output like `ForwardTransparencyPass: drew 2 transparent items (0 skipped)` 3. Put a breakpoint in `ForwardTransparencyPass::render()` (`src/engine/rendering/ForwardTransparencyPass.cpp`) and inspect the draw candidates - if the glass item never appears, the material flag or shader name is wrong; if it's skipped, pipeline creation failed (see the log)